Hardware Security Evaluation Report: Interfacing Bus Pirate with GY-MAX30102 via I2C

Engineering and Vulnerability Research Group

October 7, 2026

Contents

1 Executive Summary **2**

2 Hardware Connections and Topology **2**
2.1 Pin Interconnection Matrix 2
2.2 Interconnection Schematic 2

3 Bus Pirate Terminal Setup and Discovery **3**
3.1 Protocol Engine Configuration Mode 3
3.2 Address Space Discovery Scan 3

4 Hardware Verification and Register Manipulation **3**
4.1 Silicon Component ID Verification 3
4.2 Power Management and Mode Configuration 4

5 Reverse Engineering (RE) and Vulnerability Research (VR) **4**
5.1 Protocol Fuzzing and State Machine Subversion 4
5.2 Man-in-the-Middle (MitM) Data Injection 5
5.3 Physical Side-Channel and Counterfeit Vulnerabilities 5

1 Executive Summary

This report defines the technical implementation, physical topology, register operations, and security research profiles for evaluating the **GY-MAX30102 pulse oximeter module** using the **Bus Pirate**. The target device features a fixed 7-bit I2C peripheral address of `0x57`. When mapped to 8-bit notation for wire transmission, this corresponds to `0xAE` for write operations and `0xAF` for read operations. All protocol sequences and terminal parameters evaluated here comply strictly with the core specifications of the MAX30102 integrated circuit.

2 Hardware Connections and Topology

The GY-MAX30102 sensor module breaks out core interface lines on its PCB surface. Because the I2C protocol uses an open-drain architecture, pull-up termination is strictly required. The Bus Pirate's internal pull-up resistor network must be actively tied to the matching 3.3V rail.

2.1 Pin Interconnection Matrix

The following table outlines the exact physical connections needed between the Bus Pirate and the sensor breakout board:

Table 1: Bus Pirate to GY-MAX30102 Wiring Mapping

GY-MAX30102 Pin	Bus Pirate Pin	Signal Description	Electrical Profile
VIN (or VCC)	**3V3**	Main Power Input	3.3V DC Supply Rail
GND	**GND**	Ground Reference	Common Circuit Ground
SCL	**CLK**	I2C Serial Clock	Open-Drain Input line
SDA	**MOSI**	I2C Serial Data	Open-Drain Bidirectional line
INT	*Disconnected*	Interrupt Output	Active-Low Open-Drain (Optional)
VREG / VPU	**VPU**	Pull-Up Voltage Sense	Tied to 3V3 for pull-up source

2.2 Interconnection Schematic

Below is the system wiring visualization modeling the connection and the onboard pull-up configuration:

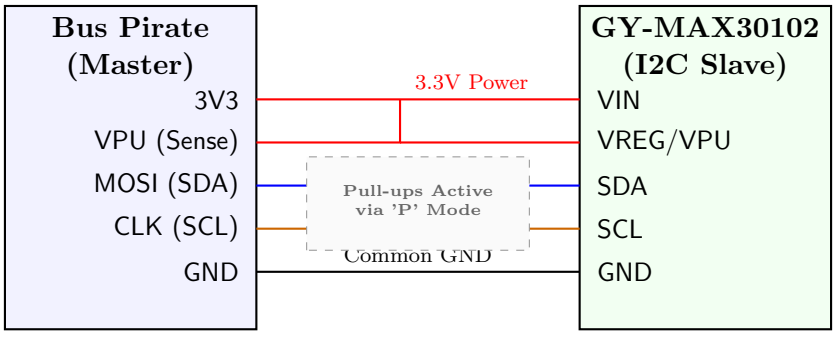


Figure 1: Hardware Wiring Diagram using Bus Pirate Pull-Up Infrastructure

3 Bus Pirate Terminal Setup and Discovery

Connect to the Bus Pirate over your serial terminal framework at **115200 bps, 8N1**. Enter the standard interactive console to change configurations.

3.1 Protocol Engine Configuration Mode

Invoke the mode selection menu (`m`), choose I2C, establish standard frequencies, and safely activate the onboard hardware rails:

```
HiZ> m          <-- Mode configuration prompt
1. HiZ
...
4. I2C          <-- Enter I2C Protocol Engine
(1)> 4
Set speed:
1. ~5KHz
2. ~50KHz
3. ~100KHz      <-- Standard Mode (Recommended for testing)
4. ~400KHz      <-- Fast Mode (Supported by MAX30102)
(1)> 3
Ready
I2C> W          <-- Capital 'W' enables the 3.3V Onboard Power Supplies
Power supplies ON
I2C> P          <-- Capital 'P' closes switches to activate VPU Pull-
   ups
Pull-up resistors ON
```

Listing 1: Initializing I2C Mode and Power Supplies

3.2 Address Space Discovery Scan

Run the built-in 7-bit automated scanner macro `(1)` to verify component presence on the line:

```
I2C> (1)
Searching I2C address space. Found devices at:
0xAE(0x57 W) 0xAF(0x57 R)   <-- Device confirmed online at address 0x57
```

Listing 2: I2C Bus Address Scan Macro Output

4 Hardware Verification and Register Manipulation

4.1 Silicon Component ID Verification

Before interacting with configuration or measurement registers, read the hardcoded **Part ID Register (`0xFF`)**. A genuine component responds with the immutable value `0x15`.

The manual command syntax sequence is constructed as follows: `[ 0xAE 0xFF [ 0xAF r ]`

- `[` : Issue an I2C Bus Start condition.
- `0xAE` : Target 8-bit Write Address (`0x57` shifted left by 1 bit, with bit 0 clear).
- `0xFF` : Send target register offset address (Part ID pointer).
- `[` : Issue an I2C Repeated Start condition.

- `0xAF` : Target 8-bit Read Address (`0x57` shifted left by 1 bit, with bit 0 set).
- `r` : Sample and retrieve one data byte from the slave target device.
- `]` : Issue an I2C Bus Stop condition.

```
I2C> [0xAE 0xFF [0xAF r]
I2C START BIT
WRITE: 0xAE ACK
WRITE: 0xFF ACK
I2C START BIT
WRITE: 0xAF ACK
READ: 0x15                  <-- Confirms genuine MAX30102 Silicon
NACK
I2C STOP BIT
```

Listing 3: Reading Part ID Verification Register

4.2 Power Management and Mode Configuration

By default, the device initializes in a low-power shutdown configuration. To awake the hardware sensor elements, update the **Mode Configuration Register (`0x09`)**:

```
I2C> [0xAE 0x09 0x03]
I2C START BIT
WRITE: 0xAE ACK
WRITE: 0x09 ACK             <-- Select Mode Configuration Register
WRITE: 0x03 ACK             <-- 0x03 sets SpO2 Mode (Active Red + IR
   Processing)
I2C STOP BIT
```

Listing 4: Waking the Sensor Module into SpO2 Mode

Note: Successful configuration immediately triggers the optical emitter arrays, causing a visible red glow on the sensor surface.

5 Reverse Engineering (RE) and Vulnerability Research (VR)

When analyzing environments that rely on this module (such as consumer fitness trackers or medical monitoring equipment), a researcher can use the Bus Pirate to assess firmware and hardware robustness.

5.1 Protocol Fuzzing and State Machine Subversion

The device uses an internal 32-sample FIFO pointer array (`0x04` to `0x07`). Researchers can build scripts to fuzz unallocated or sensitive registers:

- **Out-of-Bounds Injection:** Rapidly writing unexpected values to the **LED Pulse Amplitude Registers (`0x0C`, `0x0D`)** allows you to test whether the chip's power rails or internal state logic lock up, pull the shared data line down permanently (causing a Denial of Service), or trigger an unexpected hardware master clear.
- **FIFO Memory Exploits:** Overflowing the input buffers while manually altering read/write pointer boundaries (`0x04` and `0x06`) helps determine if the host microcontroller handles invalid pointer data gracefully or if it is susceptible to parsing errors.

5.2 Man-in-the-Middle (MitM) Data Injection

If you are evaluating an active device deployment where sensor data controls automated actions (like alert triggers), you can perform an inline bus split:

```
[Host MCU] <--- (Cut Lines) ---> [Bus Pirate Interceptor] <---> [GY-
    MAX30102]
```

Listing 5: Data Interception Architecture Flow

By intercepting and modifying packets on the fly, a researcher can spoof safe vitals metrics (e.g., maintaining a simulated SpO_2 of 98%) even if the sensor detects a drop. This allows you to evaluate how cleanly the upstream application validates data or handles protocol exceptions.

5.3 Physical Side-Channel and Counterfeit Vulnerabilities

- **Power Analysis:** Monitor current draw on the 3.3 V rail while the LEDs are pulsed. Distinct current signatures appear for different LED amplitude settings and can be used to fingerprint genuine versus counterfeit silicon.
- **Timing Side-Channels:** Measure the time between FIFO almost-full interrupt assertions and actual data availability. Variations can reveal internal state-machine behavior or manufacturing process differences.
- **Counterfeit Detection:** Compare the Part ID response, LED wavelength characteristics (via spectrometer if available), and I2C ACK timing against a known-good MAX30102 reference. Significant deviations indicate a cloned or relabeled part.